

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Muhammad Bukhari Fitri NIM. 2310817210015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Bukhari Fitri
NIM : 2310817210015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	8
B. Output Program	31
C. Pembahasan	39
D. Tautan Git	54
SOAL 2.....	55
A. Pembahasan	55

DAFTAR GAMBAR

Gambar 1. Contoh UI List	7
Gambar 2. Contoh UI Detail	7
Gambar 3. Tampilan Awal UI XML	31
Gambar 4. Tampilan Awal UI Landscape XML	32
Gambar 5. Tampilan Tombol Beli Buku Ditekan XML	32
Gambar 6. Tampilan Ketika Discroll XML	33
Gambar 7. Tampilan Tombol Detail Ditekan XML	34
Gambar 8. Tampilan Tombol Detail Ditekan Landscape XML	34
Gambar 9. Tampilan Awal UI Compose	35
Gambar 10. Tampilan Ketika Discroll Compose	36
Gambar 11. Tampilan Awal UI Landscape Compose	36
Gambar 12. Tampilan Tombol Beli Buku Ditekan Compose	37
Gambar 13. Tampilan Detail Compose	38
Gambar 14. Tampilan Detail Landscape Compose	38

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML	8
Tabel 2. Source Code Book.kt XML	8
Tabel 3. Source Code BookFragment.kt XML.....	9
Tabel 4. Source Code DetailFragment.kt XML	11
Tabel 5. Source Code MyItemRecyclerViewAdapter.kt XML	12
Tabel 6. Source Code activity_main.xml XML	14
Tabel 7. Source Code Jawaban Soal 1 activity_main XML.....	14
Tabel 8. Source Code fragment_book.xml XML	15
Tabel 9. Source Code fragment_detail.xml XML	16
Tabel 10. Source Code item_book.xml XML	19
Tabel 11. Source Code MainActivity.kt Compose.....	22
Tabel 12. Source Code Datasource.kt Compose	24
Tabel 13. Source Code Book.kt Compose.....	25
Tabel 14. Source Code BookDetailScreen.kt Compose	25
Tabel 15. Source Code BookListScreen.kt Compose.....	28

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Dusahakan agar desain UI item list menyerupai UI berikut:

A. Source Code

XML:

MainActivity.kt

Tabel 1. Source Code MainActivity.kt XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import com.example.booklist.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	val binding = ActivityMainBinding.inflate(layoutInflater)
14	setContentView(binding.root)
15	}
16	}

Book.kt

Tabel 2. Source Code Book.kt XML

1	package com.example.booklist
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(
7	@StringRes val titleResourceId: Int,
8	@StringRes val yearResourceId: Int,
9	@StringRes val aboutResourceId: Int,
10	@StringRes val webUrlResourceId: Int,
11	@DrawableRes val imageResourceId: Int
12	)

BookFragment.kt

Tabel 3. Source Code BookFragment.kt XML

```

1 package com.example.booklist
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.findNavController
9 import androidx.recyclerview.widget.LinearLayoutManager
10 import com.example.booklist.databinding.FragmentBookBinding
11
12 class BookFragment : Fragment() {
13
14     private var _binding: FragmentBookBinding? = null
15     private val binding get() = _binding!!
16
17     private val bookList = listOf(
18         Book(R.string.book_title1, R.string.book_year1,
19           R.string.book_about1, R.string.book_webUrl1,
20           R.drawable.image1),
21         Book(R.string.book_title2, R.string.book_year2,
22           R.string.book_about2, R.string.book_webUrl2,
23           R.drawable.image2),
24         Book(R.string.book_title3, R.string.book_year3,
25           R.string.book_about3, R.string.book_webUrl3,
26           R.drawable.image3),
27         Book(R.string.book_title4, R.string.book_year4,
28           R.string.book_about4, R.string.book_webUrl4,
29           R.drawable.image4),
30         Book(R.string.book_title5, R.string.book_year5,
31           R.string.book_about5, R.string.book_webUrl5,
32           R.drawable.image5),
33         Book(R.string.book_title6, R.string.book_year6,
34           R.string.book_about6, R.string.book_webUrl6,
35           R.drawable.image6),
36         Book(R.string.book_title7, R.string.book_year7,
37           R.string.book_about7, R.string.book_webUrl7,
38           R.drawable.image7),
39         Book(R.string.book_title8, R.string.book_year8,
40           R.string.book_about8, R.string.book_webUrl8,
41           R.drawable.image8)
42     )
43
44     override fun onCreateView(inflater: LayoutInflater,
45       container: ViewGroup?, savedInstanceState: Bundle?): View {
46         _binding = FragmentBookBinding.inflate(inflater,
47           container, false)
48         return binding.root
49     }
50
51     override fun onViewCreated(view: View, savedInstanceState:
52       Bundle?) {

```

34	val recyclerView = binding.recyclerView	
35	recyclerView.layoutManager	=
	LinearLayoutManager(requireContext())	
36	recyclerView.adapter = BookAdapter(requireContext(),	
	bookList) { selectedBook ->	
37	val bundle = Bundle().apply {	
38	putInt("titleRes",	
	selectedBook.titleResourceId)	
39	putInt("yearRes",	
	selectedBook.yearResourceId)	
40	putInt("aboutRes",	
	selectedBook.aboutResourceId)	
41	putInt("imageResId",	
	selectedBook.imageResourceId)	
42	}	
43	findNavController().navigate(R.id.action_bookFragment_to_detailFragment, bundle)	
44	}	
45	}	
46		
47	override fun onDestroyView() {	
48	super.onDestroyView()	
49	_binding = null	
50	}	
51	}	

DetailFragment.kt

Tabel 4. Source Code DetailFragment.kt XML

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.fragment.app.Fragment
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import com.example.booklist.databinding.FragmentDetailBinding
9	
10	class DetailFragment : Fragment() {
11	
12	private var _binding: FragmentDetailBinding? = null
13	private val binding get() = _binding!!
14	
15	override fun onCreateView(
16	inflater: LayoutInflater,
17	container: ViewGroup?,
18	savedInstanceState: Bundle?
19	): View? {
20	_binding = FragmentDetailBinding.inflate(inflater,
	container, false)

```

21         return binding.root
22     }
23
24     override fun onCreateView(view: View, savedInstanceState:
Bundle?) {
25         val args = arguments
26         val titleRes = args?.getInt("titleRes") ?: 0
27         val yearRes = args?.getInt("yearRes") ?: 0
28         val aboutRes = args?.getInt("aboutRes") ?: 0
29         val imageResId = args?.getInt("imageResId") ?: 0
30
31         binding.bookTitle.setText(titleRes)
32         binding.bookYear.setText(yearRes)
33         binding.bookAbout.setText(aboutRes)
34         binding.bookImage.setImageResource(imageResId)
35     }
36
37     override fun onDestroyView() {
38         super.onDestroyView()
39         _binding = null
40     }
41 }

```

MyItemRecyclerViewAdapter.kt

Tabel 5. Source Code MyItemRecyclerViewAdapter.kt XML

```

1 package com.example.booklist
2
3 import android.content.Context
4 import android.content.Intent
5 import android.net.Uri
6 import android.view.LayoutInflater
7 import android.view.ViewGroup
8 import androidx.recyclerview.widget.RecyclerView
9 import com.example.booklist.databinding.ItemBookBinding
10
11 class BookAdapter(
12     private val context: Context,
13     private val books: List<Book>,
14     private val onAboutClick: (Book) -> Unit
15 ) : RecyclerView.Adapter<BookAdapter.BookViewHolder>() {
16
17     inner class BookViewHolder(val binding: ItemBookBinding)
: RecyclerView.ViewHolder(binding.root)
18
19     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): BookViewHolder {
20         val binding =
ItemBookBinding.inflate(LayoutInflater.from(context), parent,
false)

```

```

21         return BookViewHolder(binding)
22     }
23
24     override fun onBindViewHolder(holder: BookViewHolder,
25 position: Int) {
26         val book = books[position]
27         with(holder.binding) {
28             bookTitle.text =
29             context.getString(book.titleResourceId)
30             bookYear.text =
31             context.getString(book.yearResourceId)
32             bookAbout.text =
33             context.getString(book.aboutResourceId)
34             bookImage.setImageResource(book.imageResourceId)
35
36             btnBuy.setOnClickListener {
37                 val intent = Intent(Intent.ACTION_VIEW,
38 Uri.parse(context.getString(book.webUrlResourceId)))
39                 context.startActivity(intent)
40             }
41
42             btnAbout.setOnClickListener {
43                 onAboutClick(book)
44             }
45         }
46
47         override fun getItemCount() = books.size
48     }

```

activity_main.xml

Tabel 6. Source Code activity_main.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<FrameLayout
11	android:id="@+id/frame_layout"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintHorizontal_bias="1.0"
17	app:layout_constraintStart_toStartOf="parent"
18	app:layout_constraintTop_toTopOf="parent"
19	app:layout_constraintVertical_bias="0.0">
20	
21	<androidx.fragment.app.FragmentContainerView
22	android:id="@+id/nav_host_fragment"
23	android:name="androidx.navigation.fragment.NavHostFragment"
24	android:layout_width="match_parent"
25	android:layout_height="match_parent"
26	app:navGraph="@navigation/nav_graph"
27	app:defaultNavHost="true" />
28	
29	</FrameLayout>
30	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 7. Source Code Jawaban Soal 1 activity_main XML

fragment_book.xml

Tabel 8. Source Code fragment_book.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".BookFragment">
9	
10	<androidx.recyclerview.widget.RecyclerView
11	android:id="@+id/recyclerView"
12	android:layout_width="0dp"
13	android:layout_height="0dp"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintBottom_toBottomOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintEnd_toEndOf="parent" />
18	
19	</androidx.constraintlayout.widget.ConstraintLayout>

fragment_detail.xml

Tabel 9. Source Code fragment_detail.xml XML

1	<LinearLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:orientation="vertical"
8	android:padding="16dp"
9	android:background="#1C1C1C">
10	<androidx.constraintlayout.widget.ConstraintLayout
11	android:layout_width="match_parent"
12	android:layout_height="match_parent">
13	
14	<androidx.cardview.widget.CardView
15	android:id="@+id/card_image"
16	android:layout_width="wrap_content"
17	android:layout_height="0dp"
18	app:cardCornerRadius="8dp"
19	app:layout_constraintEnd_toEndOf="parent"
20	app:layout_constraintStart_toStartOf="parent"
21	app:layout_constraintTop_toTopOf="parent">
22	
23	<ImageView
24	android:id="@+id/bookImage"
25	android:layout_width="350dp"
26	android:layout_height="520dp"
27	android:scaleType="centerCrop"
28	android:src="@drawable/imagel" />
29	</androidx.cardview.widget.CardView>
30	
31	<TextView
32	android:id="@+id/bookTitle"
33	android:layout_width="266dp"
34	android:layout_height="0dp"
35	android:layout_marginStart="8dp"
36	android:layout_marginTop="23dp"
37	android:layout_marginEnd="8dp"
38	android:text="@string/book_title1"
39	android:textColor="@color/white"
40	android:textSize="24sp"
41	android:textStyle="bold"
42	
43	app:layout_constraintEnd_toStartOf="@+id/bookYear"
44	app:layout_constraintStart_toStartOf="parent"
45	
46	app:layout_constraintTop_toBottomOf="@+id/card_image" />
47	<TextView
48	android:id="@+id/bookYear"
49	android:layout_width="63dp"
	android:layout_height="33dp"

```

50         android:layout_marginTop="24dp"
51         android:layout_marginEnd="8dp"
52         android:layout_marginBottom="38dp"
53         android:text="@string/book_year1"
54         android:textColor="@color/white"
55         android:textSize="24sp"
56
57     app:layout_constraintBottom_toTopOf="@+id/bookAbout"
58         app:layout_constraintEnd_toEndOf="parent"
59
60     app:layout_constraintStart_toEndOf="@+id/bookTitle"
61
62     app:layout_constraintTop_toBottomOf="@+id/card_image" />
63
64     <TextView
65         android:id="@+id/textTentang"
66         android:layout_width="61dp"
67         android:layout_height="23dp"
68         android:layout_marginStart="17dp"
69         android:layout_marginTop="8dp"
70         android:layout_marginEnd="302dp"
71         android:text="@string/about"
72         android:textColor="@color/white"
73         android:textSize="14sp"
74
75     app:layout_constraintBottom_toTopOf="@+id/bookAbout"
76         app:layout_constraintEnd_toEndOf="parent"
77         app:layout_constraintStart_toStartOf="parent"
78
79     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
80
81     <TextView
82         android:id="@+id/bookAbout"
83         android:layout_width="346dp"
84         android:layout_height="0dp"
85         android:layout_marginStart="17dp"
86         android:layout_marginEnd="16dp"
87         android:text="@string/book_about1"
88         android:textColor="@color/white"
89         android:textSize="14sp"
90         app:layout_constraintBottom_toBottomOf="parent"
91         app:layout_constraintEnd_toEndOf="parent"
92         app:layout_constraintHorizontal_bias="0.0"
93         app:layout_constraintStart_toStartOf="parent"
94
95     app:layout_constraintTop_toBottomOf="@+id/textTentang" />
96
97     </androidx.constraintlayout.widget.ConstraintLayout>
98 </LinearLayout>

```

item_book.xml

Tabel 10. Source Code item_book.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content">
7	
8	<androidx.cardview.widget.CardView
9	android:layout_width="wrap_content"
10	android:layout_height="wrap_content"
11	android:layout_margin="8dp"
12	app:cardBackgroundColor="@color/dark_gray"
13	app:cardCornerRadius="16dp"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintEnd_toEndOf="parent"
16	app:layout_constraintStart_toStartOf="parent"
17	app:layout_constraintTop_toTopOf="parent">
18	
19	<androidx.constraintlayout.widget.ConstraintLayout
20	android:layout_width="match_parent"
21	android:layout_height="wrap_content">
22	
23	<androidx.cardview.widget.CardView
24	android:id="@+id/card_image"
25	android:layout_width="wrap_content"
26	android:layout_height="wrap_content"
27	android:layout_marginStart="16dp"
28	android:layout_marginTop="16dp"
29	android:layout_marginBottom="16dp"
30	app:cardCornerRadius="8dp"
31	app:layout_constraintBottom_toBottomOf="parent"
32	app:layout_constraintStart_toStartOf="parent"
33	app:layout_constraintTop_toTopOf="parent">
34	
35	<ImageView
36	android:id="@+id/bookImage"
37	android:layout_width="123dp"
38	android:layout_height="200dp"
39	android:scaleType="centerCrop"
40	app:srcCompat="@drawable/image1" />
41	</androidx.cardview.widget.CardView>
42	
43	<TextView
44	android:id="@+id/bookTitle"
45	android:layout_width="193dp"
46	android:layout_height="wrap_content"
47	android:layout_marginTop="8dp"
48	android:layout_marginEnd="5dp"
49	android:text="@string/book_title1"

```

50         android:textColor="@color/white"
51         android:textSize="24sp"
52         android:textStyle="bold"
53     app:layout_constraintEnd_toStartOf="@+id/bookYear"
54     app:layout_constraintStart_toEndOf="@+id/card_image"
55         app:layout_constraintTop_toTopOf="parent" />
56
57     <TextView
58         android:id="@+id/bookYear"
59         android:layout_width="wrap_content"
60         android:layout_height="wrap_content"
61         android:layout_marginTop="16dp"
62         android:layout_marginEnd="16dp"
63         android:text="@string/book_year1"
64         android:textColor="@color/white"
65         app:layout_constraintEnd_toEndOf="parent"
66         app:layout_constraintTop_toTopOf="parent" />
67
68     <TextView
69         android:id="@+id/textTentang"
70         android:layout_width="wrap_content"
71         android:layout_height="0dp"
72         android:layout_marginStart="8dp"
73         android:layout_marginTop="24dp"
74         android:text="@string/about"
75         android:textColor="@color/white"
76         android:textSize="14sp"
77     app:layout_constraintEnd_toStartOf="@+id/bookAbout"
78     app:layout_constraintStart_toEndOf="@+id/card_image"
79     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
80
81     <TextView
82         android:id="@+id/bookAbout"
83         android:layout_width="184dp"
84         android:layout_height="wrap_content"
85         android:layout_marginStart="6dp"
86         android:layout_marginTop="24dp"
87         android:layout_marginEnd="9dp"
88         android:text="@string/book_about1"
89         android:textColor="@color/white"
90         android:textSize="14sp"
91         app:layout_constraintEnd_toEndOf="parent"
92     app:layout_constraintStart_toEndOf="@+id/textTentang"
93     app:layout_constraintTop_toBottomOf="@+id/bookTitle" />
94

```

95	<Button
96	android:id="@+id/btnBuy"
97	android:layout_width="wrap_content"
98	android:layout_height="wrap_content"
99	android:layout_marginStart="55dp"
100	android:layout_marginTop="27dp"
101	android:layout_marginBottom="8dp"
102	android:text="@string/beli_button"
103	android:textSize="14sp"
104	app:layout_constraintBottom_toBottomOf="parent"
105	app:layout_constraintEnd_toStartOf="@+id/btnAbout"
106	app:layout_constraintStart_toEndOf="@+id/card_image"
107	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
108	
109	<Button
110	android:id="@+id/btnAbout"
111	android:layout_width="wrap_content"
112	android:layout_height="wrap_content"
113	android:layout_marginStart="8dp"
114	android:layout_marginTop="27dp"
115	android:layout_marginEnd="16dp"
116	android:layout_marginBottom="8dp"
117	android:text="@string/detail_button"
118	android:textSize="14sp"
119	app:layout_constraintBottom_toBottomOf="parent"
120	app:layout_constraintEnd_toEndOf="parent"
121	app:layout_constraintStart_toEndOf="@+id/btnBuy"
122	app:layout_constraintTop_toBottomOf="@+id/bookAbout" />
123	</androidx.constraintlayout.widget.ConstraintLayout>
124	</androidx.cardview.widget.CardView>
125	</androidx.constraintlayout.widget.ConstraintLayout>

Compose:

MainActivity.kt

Tabel 11. Source Code MainActivity.kt Compose

1	package com.example.booklist
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent

```

6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.WindowInsets
8 import androidx.compose.foundation.layout.asPaddingValues
9 import androidx.compose.foundation.layout.calculateEndPadding
10 import
    androidx.compose.foundation.layout.calculateStartPadding
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.safeDrawing
14 import androidx.compose.foundation.layout.statusBarsPadding
15 import androidx.compose.material3.Surface
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.platform.LocalLayoutDirection
19 import androidx.compose.ui.tooling.preview.Preview
20 import androidx.compose.ui.unit.dp
21 import androidx.navigation.compose.NavHost
22 import androidx.navigation.compose.composable
23 import androidx.navigation.compose.rememberNavController
24 import com.example.booklist.data.Datasource
25 import com.example.booklist.model.Book
26 import com.example.booklist.screens.BookDetailScreen
27 import com.example.booklist.screens.BookListScreen
28 import com.example.booklist.ui.theme.BookListTheme
29
30 class MainActivity : ComponentActivity() {
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         enableEdgeToEdge()
34         setContent {
35             BookListTheme {
36                 BooksApp()
37             }
38         }
39     }
40 }
41
42 @Composable
43 fun BooksApp() {
44     val layoutDirection = LocalLayoutDirection.current
45     val navController = rememberNavController()
46     val books = Datasource().loadBooks()
47     Surface(
48         modifier = Modifier
49             .fillMaxSize()
50             .statusBarsPadding()
51             .padding(
52                 start
                    =
                    WindowInsets.safeDrawing.asPaddingValues().calculateStartPadding(layoutDirection),
53

```

	end	=
54	WindowInsets.safeDrawing.asPaddingValues().calculateEndPaddin	
55	g(layoutDirection),	
56	)	
	) {	
57	NavHost(navController	= navController,
58	startDestination = "ListScreen") {	
	composable(route = "ListScreen") {	
59	BookListScreen(bookList	= books,
	onDetailClick = {	
60	index	->
61	navController.navigate("detail/\$index")	
62	}}	
63	}	
	composable("detail/{index}") { backStackEntry ->	
64	val indexString	=
65	backStackEntry.arguments?.getString("index")	
66	val index = indexString?.toIntOrNull()	
67	val book = index?.let { books.getOrNull(it) }	
68		
69	if (book != null) {	
70	BookDetailScreen(book = book)	
	} else {	
71	androidx.compose.material3.Text("Buku	
72	tidak ditemukan", modifier = Modifier.padding(16.dp))	
73	}	
74	}	
75	}	
76	}	
	}	

Datasource.kt

Tabel 12. Source Code Datasource.kt Compose

1	package com.example.booklist.data
2	
3	import com.example.booklist.R
4	import com.example.booklist.model.Book
5	
6	class Datasource {
7	fun loadBooks(): List<Book> {
8	return listOf<Book>{
9	Book(R.string.book_title1, R.string.book_year1,
	R.string.book_about1, R.string.book_webUrl1,
	R.drawable.image1) ,
10	Book(R.string.book_title2, R.string.book_year2,
	R.string.book_about2, R.string.book_webUrl2,
	R.drawable.image2),
11	}

12	Book(R.string.book_title3, R.string.book_year3, R.string.book_about3, R.string.book_webUrl3, R.drawable.image3),
13	Book(R.string.book_title4, R.string.book_year4, R.string.book_about4, R.string.book_webUrl4, R.drawable.image4),
14	Book(R.string.book_title5, R.string.book_year5, R.string.book_about5, R.string.book_webUrl5, R.drawable.image5),
15	Book(R.string.book_title6, R.string.book_year6, R.string.book_about6, R.string.book_webUrl6, R.drawable.image6),
16	Book(R.string.book_title7, R.string.book_year7, R.string.book_about7, R.string.book_webUrl7, R.drawable.image7),
17	Book(R.string.book_title8, R.string.book_year8, R.string.book_about8, R.string.book_webUrl8, R.drawable.image8)
18	)
19	}
	}

Book.kt

Tabel 13. Source Code Book.kt Compose

1	package com.example.booklist.model
2	
3	import androidx.annotation.DrawableRes
4	import androidx.annotation.StringRes
5	
6	data class Book(7 @StringRes val titleResourceId: Int, 8 @StringRes val yearResourceId: Int, 9 @StringRes val aboutResourceId: Int, 10 @StringRes val webUrlResourceId: Int, 11 @DrawableRes val imageResourceId: Int 12)

BookDetailScreen.kt

Tabel 14. Source Code BookDetailScreen.kt Compose

1	package com.example.booklist.screens
2	
3	import android.content.res.Configuration
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.Row
7	import androidx.compose.foundation.layout.Spacer
8	import androidx.compose.foundation.layout.fillMaxWidth

```

9      import androidx.compose.foundation.layout.height
10     import androidx.compose.foundation.layout.padding
11     import androidx.compose.foundation.layout.size
12     import androidx.compose.foundation.layout.width
13     import androidx.compose.foundation.shape.RoundedCornerShape
14     import androidx.compose.material3.MaterialTheme
15     import androidx.compose.material3.Text
16     import androidx.compose.runtime.Composable
17     import androidx.compose.ui.Alignment
18     import androidx.compose.ui.Modifier
19     import androidx.compose.ui.draw.clip
20     import androidx.compose.ui.layout.ContentScale
21     import androidx.compose.ui.platform.LocalConfiguration
22     import androidx.compose.ui.platform.LocalContext
23     import androidx.compose.ui.res.painterResource
24     import androidx.compose.ui.text.style.TextOverflow
25     import androidx.compose.ui.unit.dp
26     import androidx.compose.ui.unit.sp
27     import com.example.booklist.model.Book
28
29     @Composable
30     fun BookDetailScreen(book: Book) {
31         val orientation = LocalConfiguration.current.orientation
32         val isLandscape = orientation ==
Configuration.ORIENTATION_LANDSCAPE
33
34         if (isLandscape) {
35             Row(modifier = Modifier.padding(16.dp)) {
36                 Image(
37                     painter =
painterResource(book.imageResourceId),
38                     contentDescription = null,
39                     modifier = Modifier
40                         .size(width = 220.dp, height = 350.dp)
41                         .clip(RoundedCornerShape(16.dp)),
42                     contentScale = ContentScale.Crop
43                 )
44
45                 Spacer(modifier = Modifier.width(16.dp))
46
47                 Column(
48                     modifier = Modifier.weight(1f),
49                     horizontalAlignment = Alignment.Start
50                 ) {
51                     Text(
52                         text =
LocalContext.current.getString(book.titleResourceId),
53                         style =
MaterialTheme.typography.labelLarge,
54                         fontSize = 26.sp
55                     )
56                     Text(

```

57	text	=
58	LocalContext.current.getString(book.yearResourceId),	=
59	style	=
60	MaterialTheme.typography.bodyMedium,	
61	fontSize = 20.sp	
62	)	
63	Spacer(modifier = Modifier.height(8.dp))	
64	Text("Tentang:", style	=
65	MaterialTheme.typography.bodySmall)	
66	Text(	
67	text	=
68	LocalContext.current.getString(book.aboutResourceId),	=
69	style	=
70	MaterialTheme.typography.bodySmall,	
71	fontSize = 14.sp	
72	)	
73	}	
74	}	
75	} else {	
76	Column(modifier = Modifier.padding(16.dp)) {	
77	Image(	
78	painter	=
79	painterResource(book.imageResourceId),	
80	contentDescription = null,	
81	modifier = Modifier	
82	.fillMaxWidth()	
83	.height(500.dp)	
84	.clip(RoundedCornerShape(16.dp)),	
85	contentScale = ContentScale.Crop	
86	)	
87	Spacer(modifier = Modifier.height(8.dp))	
88	Row {	
89	Text(	
90	text	=
91	LocalContext.current.getString(book.titleResourceId),	=
92	modifier = Modifier.weight(1f),	=
93	style	=
94	MaterialTheme.typography.labelLarge,	
95	fontSize = 26.sp	
96	)	
97	Text(	
98	text	=
99	LocalContext.current.getString(book.yearResourceId),	=
100	style	=
101	MaterialTheme.typography.bodyMedium,	
102	fontSize = 20.sp	
103	)	
104	}	
105	Spacer(modifier = Modifier.height(8.dp))	
106	Text("Tentang:", style	=
107	MaterialTheme.typography.bodySmall)	
108	Text(	

98	text	=
99	LocalContext.current.getString(book.aboutResourceId),	
100	style = MaterialTheme.typography.bodySmall,	
101	fontSize = 14.sp	
102	)	
103	}	
104	}	

BookListScreen.kt

Tabel 15. Source Code BookListScreen.kt Compose

1	package com.example.booklist.screens
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.Spacer
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.height
12	import androidx.compose.foundation.layout.padding
13	import androidx.compose.foundation.layout.size
14	import androidx.compose.foundation.layout.width
15	import androidx.compose.foundation.lazy.LazyColumn
16	import androidx.compose.foundation.lazy.items
17	import androidx.compose.foundation.shape.RoundedCornerShape
18	import androidx.compose.material3.Button
19	import androidx.compose.material3.Card
20	import androidx.compose.material3.MaterialTheme
21	import androidx.compose.material3.Text
22	import androidx.compose.runtime.Composable
23	import androidx.compose.ui.Alignment
24	import androidx.compose.ui.Modifier
25	import androidx.compose.ui.draw.clip
26	import androidx.compose.ui.layout.ContentScale
27	import androidx.compose.ui.platform.LocalContext
28	import androidx.compose.ui.res.painterResource
29	import androidx.compose.ui.res.stringResource
30	import androidx.compose.ui.text.style.TextOverflow
31	import androidx.compose.ui.tooling.preview.Preview
32	import androidx.compose.ui.unit.dp
33	import androidx.compose.ui.unit.sp
34	import com.example.booklist.BooksApp
35	import com.example.booklist.model.Book
36	
37	@Composable
38	

```

39 fun BookListScreen(bookList: List<Book>, onDetailClick:
40 (Int) -> Unit, modifier: Modifier = Modifier) {
41     LazyColumn(modifier = modifier) {
42         items(bookList.indices.toList()) {
43             index -> BookCard(
44                 book = bookList[index],
45                 onDetailClick = { onDetailClick(index) }
46             )
47         }
48     }
49 }
50 @Composable
51 fun BookCard(book: Book, onDetailClick: () -> Unit, modifier:
52 Modifier = Modifier) {
53     Card(
54         modifier = modifier
55         .fillMaxWidth()
56         .padding(8.dp),
57         shape = RoundedCornerShape(16.dp)
58     ) {
59         val context = LocalContext.current
60         Row(
61             modifier = Modifier
62             .padding(16.dp)
63             .fillMaxWidth()
64         ) {
65             Image(
66                 painter
67                 painterResource(book.imageResourceId),
68                 contentDescription
69                 stringResource(book.aboutResourceId),
70                 modifier = Modifier
71                 .size(width = 120.dp, height = 200.dp)
72                 .clip(RoundedCornerShape(12.dp))
73                 .align(Alignment.CenterVertically),
74                 contentScale = ContentScale.Crop
75             )
76             Spacer(modifier = Modifier.width(12.dp))
77             Column(
78                 modifier = Modifier
79                 .weight(1f)
80                 .align(Alignment.CenterVertically)
81             ) {
82                 Row(
83                     modifier = Modifier.fillMaxWidth(),
84                     horizontalArrangement
85                     Arrangement.SpaceBetween
86                 ) {
87                     Text(
88                         text
89                         context.getString(book.titleResourceId),

```

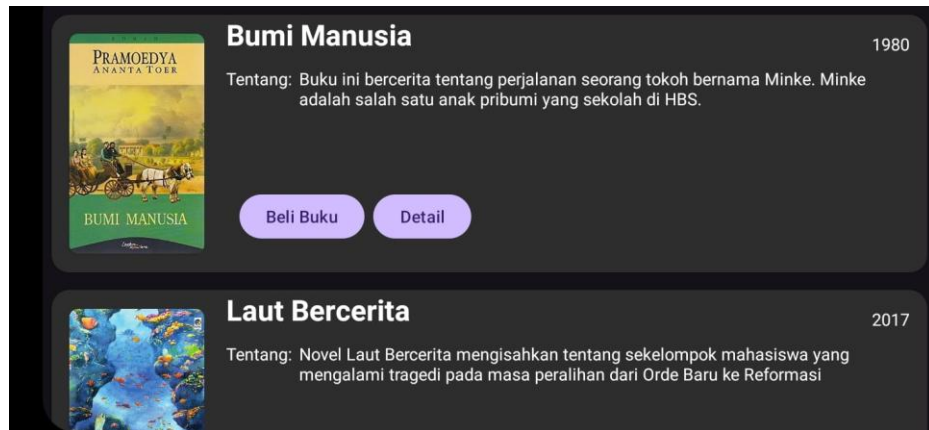
85	modifier = Modifier.weight(1f),	
86	style	=
87	MaterialTheme.typography.labelLarge,	
88	overflow = TextOverflow.Ellipsis,	
89	fontSize = 26.sp	
90	)	
91	Text(	
92	text	=
93	context.getString(book.yearResourceId),	
94	style	=
95	MaterialTheme.typography.bodyMedium,	
96	fontSize = 14.sp	
97	)	
98	}	
99	Spacer(modifier = Modifier.height(20.dp))	
100	Row {	
101	Text(	
102	text = "Tentang: ",	
103	style	=
104	MaterialTheme.typography.bodySmall,	
105	fontSize = 14.sp	
106	)	
107	Text(	
108	text	=
109	context.getString(book.aboutResourceId),	
110	style	=
111	MaterialTheme.typography.bodySmall,	
112	fontSize = 14.sp	
113	)	
114	}	
115	Spacer(modifier = Modifier.height(16.dp))	
116	Row(modifier = Modifier.fillMaxWidth(),	
117	horizontalArrangement = Arrangement.End)	
118	{	
119	Button(onClick = {	
120	val	=
121	context.getString(book.webUrlResourceId)	
122	val	=
123	Intent(Intent.ACTION_VIEW, Uri.parse(webUrl))	
124	context.startActivity(intent)	
125	}) {	
126	Text("Beli Buku")	
127	}	
	Spacer(modifier = Modifier.width(8.dp))	
	Button(onClick = {	
	onDetailClick()	
	}) {	
	Text("Detail")	
	}	
	}	

128		}
129	}	
130		
131	}	
	}	

B. Output Program



Gambar 3. Tampilan Awal UI XML



Gambar 4. Tampilan Awal UI Landscape XML



Gambar 5. Tampilan Tombol Beli Buku Ditekan XML



Pernikahan Manor milik Tuan Jones.

Beli Buku Detail

Anak Semua Bangsa

1981



Tentang: Novel Anak Semua Bangsa, Tokoh Minke, yang di dalam Bumi Manusia memiliki pola pikir kaum terdidik Belanda, mulai berubah menjadi Minke, yang sadar akan nasib kaum bumi putera.

Beli Buku Detail

Babel

2022



Tentang: Babel berpusat pada seorang tokoh utama bernama Robin Swift. Di Babel, Robin menemukan bahwa bahasa memiliki kekuatan magis yang dapat mempengaruhi dunia nyata.

Beli Buku Detail

Gambar 6. Tampilan Ketika Discroll XML



Gambar 7. Tampilan Tombol Detail Ditekan XML



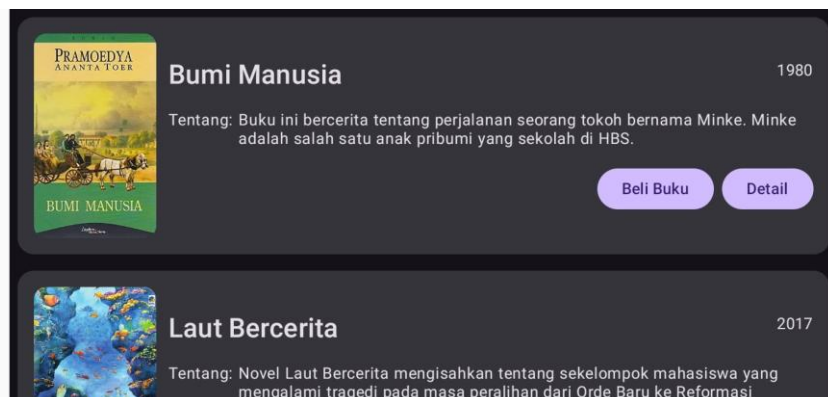
Gambar 8. Tampilan Tombol Detail Ditekan Landscape XML



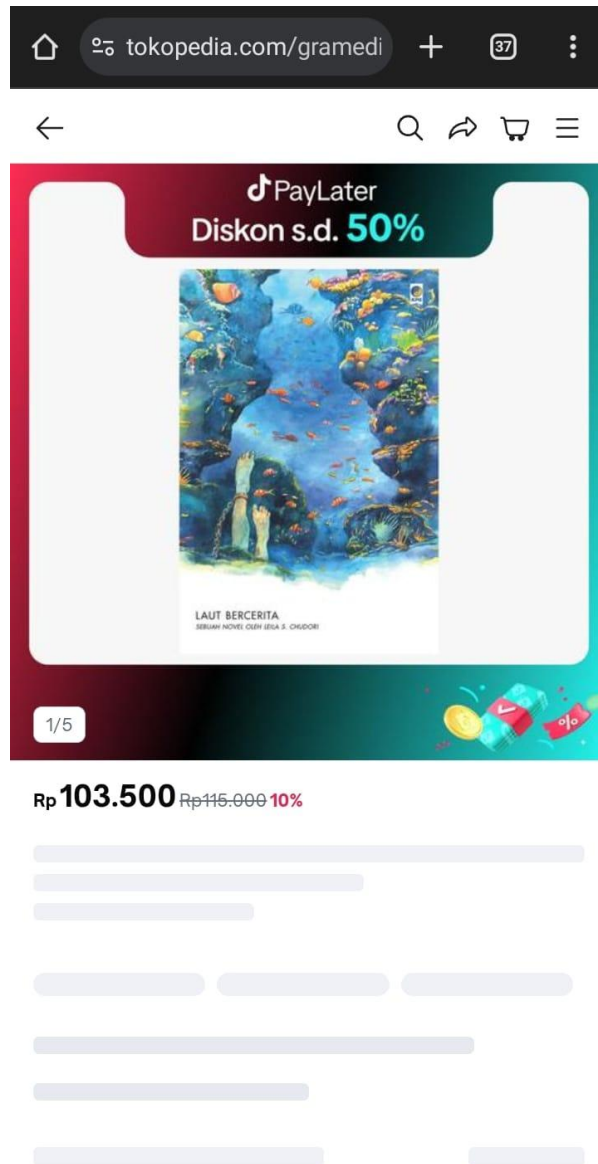
Gambar 9. Tampilan Awal UI Compose



Gambar 10. Tampilan Ketika Discroll Compose



Gambar 11. Tampilan Awal UI Landscape Compose



Gambar 12. Tampilan Tombol Beli Buku Ditekan Compose



Gambar 13. Tampilan Detail Compose



Gambar 14. Tampilan Detail Landscape Compose

C. Pembahasan

XML:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin

Pada baris [3], diimport bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada baris [4], diimport enableEdgeToEdge agar konten bisa tampil sampai ke pinggir layar.

Pada baris [5], diimport AppCompatActivity superclass untuk activity yang menggunakan fitur-fitur dari AndroidX.

Pada baris [6], diimport ViewCompat digunakan untuk manipulasi property View.

Pada baris [7], diimport WindowInsetsCompat untuk mengatur tata letak yang mempertimbangkan status bar dan navigation bar.

Pada baris [8], diimport ActivityMainBinding class binding otomatis yang dihasilkan dari file XML activity_main.xml.

Pada baris [10], mendeklarasikan kelas MainActivity yang merupakan turunan dari AppCompatActivity sebagai titik awal saat aplikasi dijalankan.

Pada baris [11], mendefinisikan method onCreate, yaitu fungsi lifecycle yang dipanggil saat activity pertama kali dibuat.

Pada baris [12], memanggil implementasi onCreate di superclass untuk memastikan sistem Android tetap menjalankan proses inisialisasi standar activity.

Pada baris [13], membuat objek binding dengan cara menginflate layout XML yaitu activity_main.xml agar bisa diakses lewat kode Kotlin.

Pada baris [14], menetapkan root dari layout yang diinflate sebagai tampilan utama activity.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3], diimport DrawableRes untuk anotasi @DrawableRes yang menandakan bahwa parameter bertipe Int tersebut adalah resource drawable atau gambar.

Pada baris [4], diimport StringRes untuk anotasi @StringRes yang menandakan bahwa parameter Int tersebut adalah resource string dari folder res/values/strings.xml.

Pada baris [6-11], mendefinisikan kelas data Book untuk menyimpan data buku. @StringRes val titleResourceId: Int adalah properti untuk menyimpan ID dari resource judul buku. Anotasi @StringRes memberi tahu Android Studio bahwa nilai ini harus berasal dari resource string (R.string.nama_judul). @StringRes val yearResourceId: Int adalah ID resource string untuk tahun terbit buku (misal "2022"). @StringRes val aboutResourceId: Int menyimpan ID resource string yang menjelaskan isi buku atau deskripsinya. @StringRes val webViewResourceId: Int menyimpan ID resource string berupa URL website (untuk beli buku). @DrawableRes val imageResourceId: Int menyimpan ID resource gambar (drawable) untuk sampul buku. Anotasi @DrawableRes menjamin bahwa nilai ini berasal dari folder res/drawable.

BookFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-10], diimport berbagai komponen penting yang dibutuhkan untuk menjalankan fungsi fragment ini. Bundle digunakan untuk mengirim data antar fragment. LayoutInflater, View, dan ViewGroup digunakan untuk mengatur tampilan layout. Fragment adalah kelas dasar untuk membuat fragment di Android. findNavController() adalah fungsi untuk melakukan navigasi antar fragment dengan Jetpack Navigation. LinearLayoutManager digunakan untuk menampilkan data list secara vertikal di dalam RecyclerView. Terakhir, FragmentBookBinding merupakan class binding otomatis yang dihasilkan dari file XML fragment_book.xml, sehingga kita bisa mengakses komponen XML-nya langsung dari Kotlin.

Pada baris [12], mendeklarasikan class fragment.

Pada baris [14], menyimpan binding dengan tipe nullable untuk menghindari memory leak.

Pada baris [15], memberi akses non null ke binding saat view aktif.

Pada baris [17-26] mendefinisikan list berisi 8 buku, setiap buku merupakan objek dari class Book yang mengambil data dari strings.xml dan gambar dari drawable.

Pada baris [28-31], method onCreateView() digunakan untuk menginflate layout fragment_book.xml menjadi view, lalu FragmentBookBinding.Inflate() menghubungkan dengan ViewBinding. Return binding.root untuk mengembalikan view utama fragment.

Pada baris [33-45], method `onViewCreated()` dipanggil setelah layout fragment berhasil ditampilkan di layar. Di dalamnya, pertama-tama diambil referensi ke `RecyclerView` menggunakan `binding.recyclerView`. Kemudian, `RecyclerView` dikonfigurasi untuk menampilkan daftar item secara vertikal dengan menggunakan `LinearLayoutManager`. Setelah itu, disetel adapter `BookAdapter`, yang akan menampilkan daftar buku dari `bookList`. Adapter juga menerima lambda (callback) yang akan dipanggil saat pengguna mengklik salah satu buku. Ketika buku diklik, dibuat `Bundle` berisi informasi judul, tahun, deskripsi, dan gambar buku. `Bundle` ini dikirimkan ke `DetailFragment` menggunakan perintah navigasi `findNavController().navigate()`.

Pada baris [47-51], method `onDestroyView()` dipanggil saat fragment akan dihancurkan (misalnya ketika berpindah ke fragment lain). Di sini, `_binding` diset ke `null` untuk menghapus referensi ke view yang sudah tidak aktif, sehingga membantu mencegah terjadinya memory leak.

DetailFragment.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–8], diimport komponen-komponen penting yang dibutuhkan dalam fragment ini. `Bundle` digunakan untuk menerima data yang dikirim dari fragment sebelumnya. `Fragment` adalah superclass yang digunakan untuk membuat fragment. `LayoutInflater`, `View`, dan `ViewGroup` digunakan untuk mengelola dan menampilkan tampilan (layout) fragment. Terakhir, `FragmentDetailBinding` adalah class `ViewBinding` yang secara otomatis dihasilkan dari file XML `fragment_detail.xml`.

Pada baris [10], dideklarasikan kelas `DetailFragment` yang merupakan turunan dari `Fragment`. Kelas ini bertugas menampilkan detail buku yang dipilih oleh pengguna dari daftar buku.

Pada baris [12], dibuat properti `_binding` yang bertipe nullable dari `FragmentDetailBinding`. Properti ini akan menyimpan objek binding yang menghubungkan komponen layout dengan kode Kotlin, dan dibuat nullable agar bisa dihapus saat view dihancurkan untuk mencegah memory leak.

Pada baris [13], dibuat properti binding yang bersifat non-nullable dengan cara memaksa akses ke `_binding` menggunakan `!!` untuk mempermudah akses ke komponen layout selama fragment masih dalam keadaan aktif.

Pada baris [15–22], method `onCreateView()` digunakan untuk menginflate layout `fragment_detail.xml` menggunakan `FragmentDetailBinding.inflate()`. Hasil inflate disimpan ke `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama dari fragment ini.

Pada baris [24–35], method `onViewCreated()` dijalankan ketika layout fragment sudah selesai ditampilkan. Di sini, kita mengambil data yang dikirim dari `BookFragment` menggunakan objek `arguments`. Kemudian, masing-masing data diambil dari `Bundle` dengan kunci `"titleRes"`, `"yearRes"`, `"aboutRes"`, dan `"imageResId"`. Jika datanya tidak tersedia, maka akan digantikan dengan nilai default 0. Setelah itu, data tersebut ditampilkan ke layout. judul buku di-set ke `bookTitle`, tahun terbit ke `bookYear`, deskripsi ke `bookAbout`, dan gambar sampul ke `bookImage`.

Pada baris [37–41], method `onDestroyView()` dipanggil saat view fragment akan dihancurkan. Di sini, `_binding` diset ke null untuk membersihkan referensi terhadap view dan mencegah terjadinya memory leak.

MyItemRecyclerViewAdapter.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–9], diimport berbagai komponen yang dibutuhkan dalam pembuatan adapter `RecyclerView`. `Context` dibutuhkan untuk akses ke resource aplikasi. `Intent` dan `Uri` digunakan untuk membuka link eksternal (seperti membuka halaman web). `LayoutInflater` dipakai untuk meng-inflate layout XML ke bentuk `View`. `ViewGroup` digunakan sebagai induk tampilan item. `RecyclerView` adalah komponen utama untuk membuat daftar yang bisa digulir. Terakhir, `ItemBookBinding` adalah class binding otomatis yang dihasilkan dari file `item_book.xml`.

Pada baris [11–15], dideklarasikan kelas `BookAdapter` sebagai turunan dari `RecyclerView.Adapter`. Kelas ini bertugas menyediakan data dan tampilan untuk setiap item buku di dalam `RecyclerView`. Adapter ini memiliki tiga parameter: `context` untuk akses ke resource dan layout inflater, `books` yaitu list data buku yang akan ditampilkan, dan

onAboutClick yaitu fungsi callback yang dijalankan ketika tombol "About" pada suatu item diklik.

Pada baris [17], dibuat inner class BookViewHolder yang bertugas menyimpan binding ke tampilan setiap item buku. Kelas ini merupakan turunan dari RecyclerView.ViewHolder, dan menggunakan binding.root sebagai tampilan utama untuk setiap elemen daftar.

Pada baris [19-22], diimplementasikan method onCreateViewHolder() yang bertugas membuat ViewHolder baru. Method ini meng-inflate layout item_book.xml menggunakan ItemBookBinding, lalu membungkusnya dalam objek BookViewHolder yang dikembalikan.

Pada baris [24-41], diimplementasikan method onBindViewHolder() yang bertugas menghubungkan data buku dengan tampilan. Data buku diambil berdasarkan posisi (index) dalam list. Kemudian, nilai dari setiap properti buku ditampilkan pada komponen UI: judul, tahun, deskripsi, dan gambar buku. Untuk tombol "Buy", ketika diklik, akan membuat Intent dengan ACTION_VIEW dan membuka URL website buku di browser. Untuk tombol "About", ketika diklik, akan memanggil fungsi onAboutClick() dengan parameter objek Book yang sesuai.

Pada baris [43], diimplementasikan method getItemCount() yang mengembalikan jumlah total item dalam list buku. Method ini digunakan oleh RecyclerView untuk mengetahui berapa banyak data yang harus ditampilkan.

activity_main.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2-30], dideklarasikan root layout yaitu ConstraintLayout, yang berasal dari library androidx.constraintlayout.widget.ConstraintLayout. Layout ini memungkinkan pengaturan posisi elemen berdasarkan batas (constraint) antar komponen atau terhadap parent. Di dalamnya juga terdapat deklarasi xmlns (XML namespace) untuk android, app, dan tools, yang masing-masing digunakan untuk atribut standar Android, atribut khusus dari AndroidX, dan preview di editor. tools:context=".MainActivity" digunakan hanya di Android Studio untuk menunjukkan bahwa layout ini digunakan oleh MainActivity.

Pada baris [10-29], didefinisikan FrameLayout dengan ID @+id/frame_layout sebagai wadah utama yang mengisi seluruh layar (match_parent untuk lebar dan tinggi). Komponen ini diberi constraint agar menempel ke semua sisi parent ConstraintLayout, yaitu Top,

Bottom, Start, dan End. Ini berarti layout akan memenuhi seluruh layar aplikasi. Biasanya horizontal dan vertikal juga diset (meskipun tidak wajib) untuk mengatur keseimbangan layout di tengah.

Pada baris [21–27], di dalam `FrameLayout`, terdapat komponen `FragmentContainerView` yang merupakan tempat untuk menampilkan fragment. Komponen ini diberi ID `@+id/nav_host_fragment`, dan diberi atribut `android:name="androidx.navigation.fragment.NavHostFragment"` yang menunjukkan bahwa fragment ini digunakan untuk sistem navigasi. Atribut `app:navGraph="@navigation/nav_graph"` menghubungkan fragment ini dengan file navigasi `nav_graph.xml`, yang berisi pengaturan alur navigasi antar fragment. Atribut `app:defaultNavHost="true"` memastikan bahwa `NavHostFragment` ini akan menangani navigasi kembali (Back) secara otomatis dari sistem.

fragment_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [2–19], dideklarasikan root layout menggunakan `ConstraintLayout` dari `AndroidX` (`androidx.constraintlayout.widget.ConstraintLayout`). Layout ini memungkinkan pengaturan posisi elemen UI secara fleksibel dengan menggunakan constraint (batas) terhadap parent atau elemen lainnya. Tiga atribut `xmlns` didefinisikan: `android` untuk atribut dasar Android, `app` untuk atribut khusus dari library `AndroidX`, dan `tools` untuk keperluan preview di `Android Studio`. Atribut `tools:context=".BookFragment"` menunjukkan bahwa layout ini digunakan oleh `BookFragment`, sehingga `Android Studio` bisa menampilkan preview sesuai konteks fragment tersebut.

Pada baris [10–17], didefinisikan sebuah `RecyclerView` dengan ID `@+id/recyclerView`. Komponen ini digunakan untuk menampilkan daftar item buku secara efisien dalam bentuk scrollable list. Lebar (`layout_width`) dan tinggi (`layout_height`) diset ke `0dp` agar ukuran akan ditentukan oleh constraint. Komponen ini diposisikan menggunakan constraint: bagian atas dan bawah ditautkan ke Top dan Bottom parent, sedangkan bagian kiri (Start) dan kanan (End) juga ditautkan ke parent, sehingga `RecyclerView` akan memenuhi seluruh ruang dalam `ConstraintLayout`.

fragment_detail.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan

Pada baris [1–92], didefinisikan LinearLayout sebagai root layout dengan orientasi vertikal. Ini berarti elemen-elemen di dalamnya akan disusun dari atas ke bawah. Layout ini mengambil seluruh lebar dan tinggi layar (match_parent), memberi padding sebesar 16dp, dan diberi latar belakang warna gelap #1C1C1C. Selain itu, terdapat deklarasi xmlns standar untuk namespace Android, atribut app khusus AndroidX, dan tools untuk keperluan preview. Pada baris [10–01], terdapat ConstraintLayout di dalam LinearLayout. Layout ini digunakan agar setiap elemen UI bisa diposisikan secara fleksibel berdasarkan constraint ke elemen lain. ConstraintLayout ini juga mengambil seluruh ruang yang tersedia di dalam parent-nya (LinearLayout).

Pada baris [14–29], terdapat CardView dengan ID card_image, yang digunakan sebagai wadah untuk menampilkan gambar buku dengan sudut melengkung (cardCornerRadius="8dp"). Tinggi layout diset 0dp karena akan ditentukan berdasarkan constraint. Di dalamnya terdapat ImageView berukuran 350dp x 520dp yang menampilkan gambar buku (@drawable/image1) dan menggunakan scaleType="centerCrop" agar gambar mengisi ruang dengan proporsi yang tepat.

Pada baris [31–44], didefinisikan TextView dengan ID bookTitle yang menampilkan judul buku. TextView ini menggunakan textSize="24sp" dan textStyle="bold", dengan warna teks putih. Letaknya dikunci menggunakan constraint ke bawah dari card_image, ke kiri parent, dan ke kanan bookYear, sehingga posisinya akan diatur antara gambar dan tahun buku.

Pada baris [46–59], terdapat TextView lain dengan ID bookYear, yang menampilkan tahun terbit buku. Ukuran teks sama seperti judul buku (24sp) dan berwarna putih. Posisi bookYear dikunci di samping kanan dari bookTitle, dan berada di bawah card_image, serta di atas bookAbout, menyesuaikan tata letak dengan fleksibel.

Pada baris [61–74], TextView dengan ID textTentang digunakan sebagai label kecil “Tentang” untuk bagian deskripsi buku. Ukurannya kecil (14sp) dan diletakkan tepat di bawah bookTitle. Atribut layout_marginEnd="302dp" membuat teks ini lebih ke kiri, menjaga proporsi dari sisi kanan.

Pada baris [76–89], TextView dengan ID bookAbout menampilkan deskripsi atau sinopsis buku. Ukuran teks kecil (14sp) dengan lebar 346dp. Komponen ini terletak di bagian paling

bawah layout dan dikunci posisinya di bawah textTentang, serta dijaga menyentuh batas bawah layar (`constraintBottom_toBottomOf="parent"`), sehingga bisa menyesuaikan tinggi layar.

Item_book.xml:

Pada baris [1], terdapat versi XML dan encoding yang digunakan.

Pada baris [2–125], layout utama menggunakan `ConstraintLayout` yang fleksibel dalam menempatkan elemen berdasarkan constraint. Lebarnya diset `match_parent`, artinya akan mengisi lebar parent-nya, dan tingginya `wrap_content`, jadi hanya sebesar konten di dalamnya.

Pada baris [8–24], terdapat `CardView` sebagai kontainer utama yang membungkus semua isi item buku. Ini memberikan efek melengkung pada sudutnya (`cardCornerRadius="16dp"`) dan latar belakang abu gelap (`@color/dark_gray`). `CardView` ini diberi margin 8dp agar tidak mepet ke item lain di `RecyclerView`. Constraint-nya mengatur agar tetap di tengah-tengah parent layout.

Pada baris [19–123], di dalam `CardView` luar terdapat `ConstraintLayout` kedua yang mengatur elemen-elemen detail buku.

Pada baris [23–33], terdapat `CardView` kecil (ID: `card_image`) yang berisi gambar buku (`ImageView`). `CardView` ini memiliki `cardCornerRadius="8dp"` agar gambar memiliki sudut melengkung. Di dalamnya, `ImageView` berukuran 123dp x 200dp menampilkan gambar buku dengan `scaleType="centerCrop"` agar gambar memenuhi ruang secara proporsional.

Pada baris [43–55], `TextView` dengan ID `bookTitle` menampilkan judul buku. Ukurannya besar (24sp), bold, dan berwarna putih. Constraint-nya menempatkan teks ini di kanan gambar, dan tidak sampai ke tepi kanan karena akan ada elemen lain di sana.

Pada baris [57–66], `TextView` dengan ID `bookYear` menampilkan tahun terbit buku, berada di kanan `bookTitle`, dan dekat tepi kanan parent.

Pada baris [68–79], `TextView` dengan ID `textTentang` menampilkan label “Tentang”. Letaknya tepat di bawah `bookTitle`, dan berada di sebelah kiri `bookAbout`.

Pada baris [81–93], `TextView` dengan ID `bookAbout` berisi sinopsis atau deskripsi buku. Letaknya di sebelah kanan `textTentang`, dan posisinya di bawah `bookTitle`. Lebar ditentukan 184dp, agar cukup panjang untuk menampilkan kalimat pendek atau potongan sinopsis.

Pada baris [95–107], terdapat Button dengan ID btnBuy untuk aksi “Beli”. Tombol ini posisinya di bawah bookAbout dan di sebelah kiri tombol detail. Ukurannya fleksibel (wrap_content) dengan teks dari string resource beli_button.

Pada baris [109–122], tombol btnAbout untuk membuka detail buku. Letaknya di sebelah kanan tombol beli, dan juga di bawah bookAbout. Tombol ini akan memicu perpindahan ke halaman detail jika diklik.

Compose:

MainActivity.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–28], diimpor berbagai komponen yang diperlukan untuk aktivitas dan tampilan aplikasi. ComponentActivity diimpor untuk membuat activity berbasis komponen, yang digunakan untuk menginisialisasi dan mengatur UI dalam aplikasi Android menggunakan Jetpack Compose. setContent digunakan untuk menetapkan tampilan UI aplikasi, sementara enableEdgeToEdge memungkinkan tampilan aplikasi untuk menggunakan seluruh area layar, mengabaikan area tepi yang biasanya terblokir oleh status bar atau navigation bar.

Pada baris [30], MainActivity dideklarasikan sebagai turunan dari ComponentActivity. Kelas ini merupakan titik awal aplikasi yang dijalankan ketika aplikasi dibuka. Sebagai aktivitas utama, MainActivity bertanggung jawab untuk mengatur siklus hidup aplikasi dan menampilkan antarmuka pengguna menggunakan Jetpack Compose.

Pada baris [31-39], dalam method onCreate(), kita memanggil super.onCreate(savedInstanceState) untuk memastikan siklus hidup activity dijalankan sesuai dengan standar. Kemudian, enableEdgeToEdge() dipanggil untuk memungkinkan konten tampilan memenuhi seluruh layar, tanpa ada pembatasan dari status bar. Selanjutnya, setContent digunakan untuk menetapkan tampilan UI aplikasi yang menggunakan tema BookListTheme, yang didefinisikan dalam file tema dan memastikan konsistensi gaya visual aplikasi. Di dalamnya, kita memanggil fungsi BooksApp() yang akan menampilkan tampilan utama aplikasi.

Pada baris [42-76], terdapat deklarasi fungsi BooksApp(), yang bertanggung jawab untuk mengelola struktur UI utama aplikasi dengan menggunakan Jetpack Compose. Fungsi ini

diatur agar dapat bekerja dengan sistem navigasi dan memberikan tampilan daftar buku dan detail buku yang dapat dipilih oleh pengguna.

Pada baris [44], variabel `layoutDirection` dideklarasikan untuk mendapatkan informasi tentang arah tata letak UI. Variabel ini berguna untuk mengelola arah tata letak komponen, terutama ketika mendukung bahasa RTL (Right-to-Left), yang mempengaruhi cara elemen-elemen UI diposisikan.

Pada baris [45], variabel `navController` dideklarasikan dan diinisialisasi dengan `rememberNavController()`, yang menyediakan kontrol untuk navigasi antar layar dalam aplikasi. `navController` digunakan untuk mengelola transisi antar tampilan atau komponen dalam aplikasi, seperti berpindah dari tampilan daftar buku ke tampilan detail buku.

Pada baris [46], variabel `books` diinisialisasi dengan memanggil `Datasource().loadBooks()`. Di sini, kita memanggil method `loadBooks()` dari class `Datasource` untuk mengambil data buku yang akan ditampilkan dalam aplikasi. Data buku ini berupa list objek `Book` yang mencakup informasi seperti judul, tahun terbit, deskripsi, dan gambar.

Pada baris [47–74], `Surface` dideklarasikan sebagai komponen pembungkus untuk menampilkan konten UI, dengan menggunakan modifier untuk mengatur ukuran dan padding. `fillMaxSize()` membuat `Surface` mengambil seluruh ruang yang tersedia di layar, sementara `statusBarsPadding()` memberi ruang di bagian atas layar agar konten tidak tertutup oleh status bar. Kemudian, `padding()` digunakan untuk menambahkan padding di sisi kiri dan kanan berdasarkan tata letak yang aman (*safe drawing area*), yang memastikan konten tidak terhalang oleh elemen UI seperti status bar atau navigation bar.

Pada baris [56–73], `NavHost` dideklarasikan untuk mengelola navigasi antar tampilan. `startDestination` ditetapkan ke `"ListScreen"`, yang berarti aplikasi akan memulai dengan menampilkan tampilan daftar buku. Di dalam `NavHost`, ada dua route yang didefinisikan. Route `"ListScreen"`, `BookListScreen` ditampilkan, dengan daftar buku yang diambil dari `books`. Ketika sebuah item buku diklik, fungsi `onDetailClick` akan menavigasi ke tampilan detail dengan mengirimkan parameter `index` buku yang dipilih. Route `"detail/{index}"`, tampilan `BookDetailScreen` ditampilkan berdasarkan `index` yang diterima dari route. Jika `index` valid, maka detail buku yang sesuai akan ditampilkan. Jika tidak, sebuah pesan "Buku tidak ditemukan" akan ditampilkan menggunakan `Text`.

Pada baris [62–72], `composable("detail/{index}")` digunakan untuk mendefinisikan tampilan detail buku berdasarkan parameter `index`. Parameter ini diterima melalui `backStackEntry`, yang berisi informasi navigasi seperti argumen yang diteruskan. `indexString` adalah nilai yang diterima dari route sebagai string, dan kemudian dikonversi menjadi integer menggunakan `toIntOrNull()`. Jika konversi berhasil, buku yang sesuai akan ditampilkan, jika tidak, pesan error "Buku tidak ditemukan" akan ditampilkan.

Datasource.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-4], diimport resource R yang merujuk pada file resource aplikasi, seperti `strings.xml` dan `drawable`. Import ini memungkinkan kita untuk mengakses berbagai resource yang ada dalam aplikasi, seperti string untuk teks, gambar untuk `drawable`, dan lainnya. Selain itu, diimport juga `Book` dari model `com.example.booklist.model.Book`. `Book` adalah class model yang menyimpan informasi tentang buku, seperti judul, tahun terbit, deskripsi, URL, dan gambar sampul yang akan digunakan dalam aplikasi untuk menampilkan data buku.

Pada baris [6], dideklarasikan class `Datasource`, yang bertugas untuk menyediakan data buku. Class ini merupakan bagian dari arsitektur aplikasi yang bertanggung jawab untuk mengelola dan menyediakan data yang diperlukan. `Datasource` biasanya digunakan untuk memisahkan logika pengambilan data dari logika tampilan atau UI, sehingga kode aplikasi lebih modular dan mudah dikelola. Dalam hal ini, class `Datasource` menyediakan data buku yang akan digunakan oleh fragment atau activity dalam aplikasi.

Pada baris [7–18], terdapat method `loadBooks()` yang berfungsi untuk mengembalikan sebuah list yang berisi objek `Book`. Dalam method ini, kita menggunakan `listOf<Book>()` untuk membuat sebuah list baru yang berisi objek-objek `Book`. Setiap objek `Book` dibentuk dengan memasukkan berbagai ID resource dari file `strings.xml` dan `drawable`. Misalnya, untuk judul buku, kita menggunakan `R.string.book_title1` yang merujuk pada string dengan nama `book_title1` di `strings.xml`. Begitu juga dengan tahun terbit, deskripsi, URL, dan gambar sampul buku yang diambil dari file `strings.xml` dan folder `drawable`. Dengan cara ini, aplikasi kita menjadi lebih fleksibel karena kita bisa mengelola data di file resource dan tidak perlu mengubah kode sumber setiap kali ada perubahan pada data buku. List yang berisi data

buku ini kemudian dapat digunakan oleh komponen lain dalam aplikasi, seperti adapter di RecyclerView, untuk menampilkan daftar buku ke pengguna.

Book.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3–4], diimpor dua anotasi penting dari AndroidX: @DrawableRes dan @StringRes. Kedua anotasi ini digunakan untuk menunjukkan bahwa nilai parameter merupakan ID dari resource, bukan data literal.

Pada baris [6–12], didefinisikan sebuah data class bernama Book. Class ini digunakan untuk merepresentasikan data sebuah buku dalam bentuk struktur objek yang sederhana dan otomatis menyediakan fungsi-fungsi dasar seperti toString(), equals(), dan hashCode() karena merupakan data class. titleResourceId, yearResourceId, aboutResourceId, dan webViewResourceId bertipe Int dan diberi anotasi @StringRes, menandakan bahwa nilainya merujuk pada string resource. Sedangkan imageResourceId bertipe Int dan diberi anotasi @DrawableRes, menandakan bahwa ini adalah ID dari resource gambar.

BookDetailScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-27], diimport komponen penting dari Android dan Jetpack Compose. Intent dan Uri dari android.content dan android.net digunakan untuk membuka URL eksternal. Komponen tata letak seperti Row, Column, Spacer, Image, dan LazyColumn diambil dari androidx.compose.foundation.layout dan foundation.lazy. Untuk tampilan material modern, digunakan Card, Button, Text, dan MaterialTheme dari androidx.compose.material3. Elemen tambahan seperti painterResource, stringResource, LocalContext, serta atribut tampilan (clip, ContentScale, dan Alignment) mendukung pengambilan resource dan pengaturan gambar. Preview, Modifier, dan unit ukuran seperti dp dan sp juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data Book diimpor dari package model dan class BooksApp disiapkan dari package utama aplikasi.

Pada baris [30-104], dideklarasikan fungsi BookDetailScreen() sebagai composable yang menerima satu parameter book bertipe Book. Fungsi ini bertugas menampilkan halaman detail dari sebuah buku tertentu, termasuk gambar, judul, tahun, dan deskripsi buku.

Pada baris [31-32], memanggil `LocalConfiguration.current.orientation` untuk mengetahui orientasi perangkat. Nilai ini kemudian disimpan dalam variabel `orientation`. Selanjutnya dibuat variabel `isLandscape`, yang bernilai `true` jika orientasi adalah `Configuration.ORIENTATION_LANDSCAPE`. Jika orientasi layar adalah landscape, maka blok kode di dalam kondisi `if (isLandscape)` akan dijalankan, dimulai dari baris [34].

Pada baris [35-43], digunakan layout `Row`, yang menata elemen secara horizontal. Di dalamnya terdapat `Image` yang menampilkan gambar buku berdasarkan resource ID yang disimpan dalam objek `book`. Fungsi `painterResource(book.imageResourceId)` digunakan untuk memuat gambar `drawable`. Modifier seperti `size(width = 220.dp, height = 350.dp)` mengatur ukuran gambar, dan `clip(RoundedCornerShape(16.dp))` memberikan sudut yang membulat. `contentScale = ContentScale.Crop` memastikan gambar memenuhi ukuran yang ditentukan tanpa distorsi.

Pada baris [45], `Spacer(modifier = Modifier.width(16.dp))` digunakan untuk memberi jarak horizontal antara gambar dan kolom teks di sebelahnya.

Pada baris [47-69], digunakan `Column` untuk menampilkan teks dalam bentuk vertikal. `Modifier.weight(1f)` pada kolom ini mengatur agar kolom mengisi ruang kosong yang tersedia. Properti `horizontalAlignment = Alignment.Start` mengatur agar semua elemen dalam kolom diratakan ke kiri.

Pada baris [51-55], teks judul buku ditampilkan menggunakan `Text`. Nilai teks diambil menggunakan `LocalContext.current.getString(book.titleResourceId)` yang menerjemahkan ID string dari resource menjadi teks nyata. Gaya teks diatur menggunakan `MaterialTheme.typography.labelLarge`, dan ukuran huruf ditetapkan sebesar 26.sp.

Pada baris [56-60], tahun terbit buku ditampilkan dengan pendekatan yang serupa, menggunakan `Text` dan mengambil nilai dari `book.yearResourceId`, lalu ditampilkan dengan gaya `bodyMedium` dan ukuran 20.sp.

Pada baris [61], `Spacer` menambahkan jarak vertikal sebesar 8.dp sebelum bagian deskripsi. Pada baris [62], teks “Tentang:” ditampilkan sebagai label sebelum deskripsi buku, menggunakan gaya `bodySmall` dari tema `Material Design`.

Pada baris [63-66], deskripsi buku ditampilkan dalam `Text`, dengan teks yang diambil dari `book.aboutResourceId`. Ukuran teks ditetapkan 14.sp, dan gaya teks tetap menggunakan

bodySmall. Jika orientasi perangkat adalah portrait (bukan landscape), maka blok else mulai dari baris [70] akan dieksekusi.

Pada baris [71–80], digunakan Column sebagai layout utama. Gambar buku ditampilkan dengan lebar penuh (fillMaxWidth()) dan tinggi 500.dp. Modifier clip(RoundedCornerShape(16.dp)) tetap digunakan untuk menghasilkan tampilan gambar yang membulat, serta ContentScale.Crop memastikan gambar tetap proporsional.

Pada baris [81], Spacer kembali ditambahkan untuk memberi jarak antara gambar dan teks.

Pada baris [82–94], Row digunakan untuk menampilkan judul buku dan tahun terbit secara horizontal. Text pertama menampilkan judul, diberi modifier weight(1f) agar mengisi ruang tersisa, dengan ukuran huruf 26.sp. Di sisi kanan, tahun terbit ditampilkan menggunakan gaya teks bodyMedium dengan ukuran 20.sp.

Pada baris [95], jarak vertikal ditambahkan dengan Spacer sebelum teks deskripsi.

Pada baris [96], label teks “Tentang:” ditampilkan menggunakan gaya teks kecil (bodySmall).

Pada baris [97–101], ditampilkan deskripsi lengkap buku, diambil dari resource string melalui LocalContext.current.getString(book.aboutResourceId), dengan ukuran font 14.sp dan gaya teks bodySmall.

BookListScreen.kt:

Pada baris [1], dideklarasikan nama package file Kotlin.

Pada baris [3-35], diimport komponen penting dari Android dan Jetpack Compose. Intent dan Uri dari android.content dan android.net digunakan untuk membuka URL eksternal. Komponen tata letak seperti Row, Column, Spacer, Image, dan LazyColumn diambil dari androidx.compose.foundation.layout dan foundation.lazy. Untuk tampilan material modern, digunakan Card, Button, Text, dan MaterialTheme dari androidx.compose.material3. Elemen tambahan seperti painterResource, stringResource, LocalContext, serta atribut tampilan (clip, ContentScale, dan Alignment) mendukung pengambilan resource dan pengaturan gambar. Preview, Modifier, dan unit ukuran seperti dp dan sp juga digunakan untuk mendukung tampilan dan pengujian UI. Terakhir, data Book diimpor dari package model dan class BooksApp disiapkan dari package utama aplikasi.

Pada baris [37-47], didefinisikan fungsi composable BookListScreen() yang menerima tiga parameter: bookList berupa daftar buku (List<Book>), onDetailClick yaitu lambda untuk menangani klik tombol detail, dan modifier opsional untuk menyesuaikan tampilan dari luar. Pada baris [39], digunakan LazyColumn sebagai layout vertikal yang mendukung pemrosesan daftar besar secara efisien. Modifier disisipkan jika diperlukan dari luar composable ini.

Pada baris [40-45], digunakan fungsi items() untuk mengiterasi daftar indeks dari bookList. Untuk setiap index, dipanggil composable BookCard, di mana objek bookList[index] dikirim sebagai argumen. Parameter onDetailClick juga diteruskan, membungkus nilai index agar dapat dikenali saat tombol diklik.

Pada baris [50-131], dideklarasikan composable BookCard(), yang bertugas menampilkan kartu berisi data singkat dari sebuah buku. Fungsi ini menerima book sebagai parameter utama, onDetailClick untuk navigasi ke detail, dan modifier opsional.

Pada baris [51-130], digunakan Card dari Material 3 sebagai kontainer utama yang memiliki padding sebesar 8.dp dan bentuk sudut membulat (RoundedCornerShape 16.dp). Modifier fillMaxWidth() memastikan kartu mengisi lebar layar.

Pada baris [57], variabel context diset sebagai LocalContext.current agar dapat digunakan untuk mengambil string resource atau memulai intent dari dalam fungsi composable.

Pada baris [58-127], layout Row digunakan untuk menyusun elemen secara horizontal. Padding ditetapkan 16.dp, dan fillMaxWidth() membuat baris mengisi lebar penuh.

Pada baris [63-71], ditampilkan gambar buku menggunakan Image. Resource gambar diakses dari ID dalam book.imageResourceId melalui painterResource. contentDescription diberikan melalui stringResource(book.aboutResourceId) agar sesuai dengan prinsip aksesibilitas. Ukuran gambar diatur dengan size(width = 120.dp, height = 200.dp) dan bentuknya dibulatkan menggunakan clip(RoundedCornerShape(12.dp)). Gambar disejajarkan secara vertikal di tengah baris (align(Alignment.CenterVertically)) dan ContentScale.Crop menjaga rasio aspek gambar.

Pada baris [72], Spacer memberikan jarak horizontal 12.dp antara gambar dan kolom teks di sebelah kanan.

Pada baris [73-127], digunakan Column untuk menampilkan isi teks dan tombol dalam tata letak vertikal. Modifier weight(1f) membuat kolom ini memenuhi ruang horizontal yang

tersisa, dan `align(Alignment.CenterVertically)` memastikan kolom berada di tengah baris secara vertikal.

Pada baris [78–95], `Row` di dalam kolom digunakan untuk menampilkan judul buku di sebelah kiri dan tahun terbit di kanan. `Arrangement.SpaceBetween` memastikan keduanya memiliki jarak yang merata di antara mereka. Judul buku ditampilkan dalam `Text` menggunakan resource string, dengan gaya `labelLarge`, ukuran huruf 26.sp, dan `overflow = TextOverflow.Ellipsis` agar teks panjang tidak meluber keluar. Tahun terbit ditampilkan di kanan dengan ukuran lebih kecil, 14.sp, dan gaya `bodyMedium`.

Pada baris [96], `Spacer` menambahkan jarak vertikal sebesar 20.dp sebelum deskripsi buku.

Pada baris [97–108], terdapat `Row` untuk memuat deskripsi buku ditampilkan dalam bentuk dua `Text`. Pertama teks label “Tentang:”, lalu diikuti oleh isi deskripsi dari resource string `book.aboutResourceId`. Keduanya menggunakan gaya `bodySmall` dan ukuran huruf 14.sp.

Pada baris [109], `Spacer` memberikan jarak sebesar 16.dp sebelum tombol aksi.

Pada baris [110–125], digunakan `Row` dengan `horizontalArrangement = Arrangement.End` agar tombol-tombol sejajar di sebelah kanan layar. Dua buah tombol ditampilkan: “Beli Buku” dan “Detail”.

Pada baris [112–118], tombol “Beli Buku” diberi aksi ketika diklik, yaitu mengambil URL dari `book.webUrlResourceId`, lalu membuat `Intent` dengan `ACTION_VIEW` dan membuka URL tersebut dengan `context.startActivity()` untuk membuka halaman pembelian buku melalui browser.

Pada baris [119], `Spacer` memberi jarak horizontal antara dua tombol sebesar 8.dp.

Pada baris [120–124], tombol “Detail” memanggil `onDetailClick()`, yaitu fungsi yang diteruskan dari luar `composable` dan biasanya mengarahkan pengguna ke halaman detail buku.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

[Praktikum Mobile/Modul 3 at main · Bukhrii/Praktikum Mobile](#)

SOAL 2

Soal Praktikum:

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

Alasan kenapa RecyclerView masih digunakan walaupun LazyColumn lebih sederhana, karena RecyclerView support banyak fitur dan jauh lebih fleksibel karena imperative. Selain itu, masih kompatibel dengan project lama (masih bisa dipakai untuk pengembang yang tidak mau bermigrasi ke Jetpack Compose). Sehingga, RecyclerView lebih cocok untuk aplikasi yang kompleks dan dinamis karena termasuk paradigma imperative (mengatur logikanya sendiri), sedangkan LazyColumn cocok untuk list simple karena termasuk pendekatan declarative (cukup sebutkan apa yang diinginkan tanpa membuat logikanya sendiri sehingga sulit diotak atik)